REPORT
ON
AFNO-BASED FOURCASTNET MODEL FOR ANTARCTIC WEATHER FORECASTING

BY

Name of the student:	ID No.:	Discipline:
Shaswat Sahoo	2024A7PS0152H	B.E. in CSE

Prepared in fulfillment of the
Practice School – I

AT

National Centre for Polar and Ocean Research (NCPOR), Goa
A Practice School – I
Station of

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

July, 2026

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

Practice School Division

Station	National Centre for Polar and Ocean Research, Goa
Duration	May 2026 – July 2026
Date of start	25th May 2026
Date of submission	18th July 2026
Title of the project:	AFNO-Based FourCastNet Model
ID No.(s), Name(s) and Disciplines:	2024A7PS0152H, Shaswat Sahoo, B.E. CSE
Name(s) and Designation(s) of expert(s):	Prof. VS Samy, Scientist, NCPOR
Name(s) of PS faculty member(s):	Prof. Hemant Rathore
Key words:	FourCastNet, AFNO, Antarctic, ERA5, Deep Learning, Weather Forecasting, Bias Correction
Project area(s):	Numerical Weather Prediction, Deep Learning, Polar Meteorology
Abstract:	Lightweight AFNO-based FourCastNet model for regional Antarctic weather forecasting at Maitri and Bharati stations. ERA5 reanalysis data (2016–2026) is used with a 48h input window to produce 24h forecasts at 6-hourly resolution. Post-processing bias correction using Ridge Regression achieves an out-of-sample temperature R^2 of 0.918 for Maitri.

Signature of the student(s)
Date:

Signature of the PS faculty member(s)
Date:

Acknowledgements

I would like to express my sincere gratitude to the National Centre for Polar and Ocean Research (NCPOR), Goa, for providing me with the opportunity to undertake this Practice School internship in such a stimulating and scientifically rich environment. The experience of working on Antarctic weather forecasting has been both intellectually enriching and personally rewarding.

I am deeply grateful to my expert mentor at NCPOR, Prof. VS Samy, for their invaluable guidance, patient explanations, and continuous encouragement throughout the project. Their domain expertise in polar meteorology and numerical weather prediction provided indispensable direction to this work.

I also thank the Practice School faculty member, Prof. Hemant Rathore, for their academic oversight and constructive feedback during the course of this internship.

I acknowledge the European Centre for Medium-Range Weather Forecasts (ECMWF) and the Copernicus Climate Data Store (CDS) for making the ERA5 reanalysis dataset publicly available, which formed the backbone of this project. I also thank the developers of the original FourCastNet framework whose open-source contributions made this regional adaptation possible.

Finally, I thank BITS Pilani, Hyderabad Campus, and the Practice School Division for designing the PS-I programme, which bridges academic learning with real-world scientific challenges.

Signature of the student

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Weather forecasting in polar regions presents unique scientific and operational challenges. The Antarctic continent, characterised by extreme cold, strong katabatic winds, and limited observational infrastructure, remains one of the most data-sparse regions on Earth. Accurate meteorological prediction over Antarctica is critical for the safety of research personnel, logistics planning of polar expeditions, and understanding of global climate systems.

Traditional Numerical Weather Prediction (NWP) systems such as ECMWF's Integrated Forecasting System (IFS) rely on physics-based dynamical models that are computationally intensive and struggle to represent fine-scale polar phenomena. In recent years, data-driven deep learning models have emerged as powerful alternatives, demonstrating skill comparable to or exceeding operational NWP systems at a fraction of the computational cost [14].

FourCastNet (Fourier Forecasting Neural Network), developed by Pathak et al. (2022) at NVIDIA, represents a landmark in this direction. It employs the Adaptive Fourier Neural Operator (AFNO) as its core computational block, enabling efficient modelling of global atmospheric dynamics by operating in the Fourier frequency domain. FourCastNet demonstrated competitive skill against ECMWF's IFS on global forecasting benchmarks while being orders of magnitude faster at inference.

This project adapts the FourCastNet philosophy to a regional Antarctic context. A lightweight AFNO-based model is trained on ERA5 reanalysis data over the Antarctic domain, with special attention to two permanent Indian research stations operated by NCPOR: Maitri (Schirmacher Oasis) and Bharati (Larsemann Hills).

1.1 Objectives

The primary objectives of this project are:

1. To acquire, preprocess, and structure ERA5 reanalysis data over the regional Antarctic domains for deep learning-based training.

2. To implement a lightweight, resolution-flexible AFNO-based FourCastNet architecture.

3. To train the model to produce 24-hour forecasts from a 48-hour input history at 6-hourly resolution.

4. To design a thermal-safe CPU-only training pipeline that runs reliably on local computing hardware.

5. To develop a post-processing bias correction layer using Ridge Regression and cyclical time features to correct systematic temperature and wind speed forecast biases.

6. To evaluate forecast quality through diagnostic visualizations and validation dashboards on out-of-sample data.

1.2 Organisation of the Report

Chapter 2 provides a literature review of relevant work in data-driven weather forecasting and neural operators. Chapter 3 describes the dataset and preprocessing methodology. Chapter 4 presents the model architecture in detail. Chapter 5 covers training configuration and post-midsem modifications. Chapters 6 and 7 cover the validation results and bias correction for Maitri station. Chapter 8 summarizes the pipeline for Bharati station. Chapter 9 presents a head-to-head comparison of FourCastNet with other major models. Chapter 10 compiles all visual dashboards and validation figures in a comprehensive catalog. Chapters 11 and 12 detail conclusions, future work, and recommendations.

Chapter 2

Literature Review

2.1 Data-Driven Weather Prediction

The application of deep learning to weather forecasting has a long history, but the emergence of global-scale transformer-based models marks a qualitative shift. Weyn et al. (2020) demonstrated the viability of convolutional neural networks for medium-range forecasting on a cubed-sphere grid [19], while Rasp and Thuerey (2021) showed that purely data-driven models could begin to approach NWP skill levels [16].

Pangu-Weather [2], developed by Huawei, employs a 3D Earth Transformer architecture to achieve superior 10-day forecast accuracy compared to ECMWF's operational model. GraphCast [11], developed by Google DeepMind, uses a graph neural network over a multi-scale icosahedral mesh and achieves state-of-the-art skill on 1,380 forecast targets. Aurora [4] further pushes the boundary with a large foundation model approach. These works collectively establish that data-driven models can serve as viable operational forecasting tools.

2.2 Fourier Neural Operators and FourCastNet

Neural Operators represent a paradigm for learning mappings between function spaces, enabling resolution-independent generalisation. The Fourier Neural Operator (FNO) proposed by Li et al. (2021) applies spectral convolutions in the Fourier domain to learn PDE solution operators efficiently [12]. The Adaptive Fourier Neural Operator (AFNO), introduced by Guibas et al. (2021), extends this to vision transformer-style architectures [18] with learned Fourier mixing [5].

FourCastNet [14] integrates AFNO blocks into an end-to-end autoregressive forecasting architecture trained on ERA5. It demonstrated a 45,000× speedup over ECMWF IFS at inference while matching forecast skill at 2-week lead times for many variables. Its open-source release made regional adaptation projects such as the present one feasible.

2.3 Antarctic Meteorological Modelling

Antarctic weather forecasting has historically relied on global NWP output supplemented by regional atmospheric models such as the Polar WRF [3, 17] and AMPS

(Antarctic Mesoscale Prediction System) [15]. These systems, while physically principled, are computationally expensive and require regular updates to boundary conditions from global models.

The limited observational density over Antarctica – few radiosonde stations, sparse surface networks, and challenging satellite retrieval – makes data assimilation difficult. Deep learning models trained on reanalysis data offer a promising alternative for regional deployment, particularly for research stations such as Maitri and Bharati operated by NCPOR.

Chapter 3

Dataset and Preprocessing

3.1 ERA5 Reanalysis Data

The ERA5 reanalysis dataset, produced by the European Centre for Medium-Range Weather Forecasts (ECMWF), serves as the primary data source for this project. ERA5 provides global atmospheric data at 0.25° horizontal resolution with hourly temporal resolution, assimilating observations from satellites, radiosondes, surface stations, and aircraft through a state-of-the-art 4D-Var data assimilation system [8].

3.2 Maitri vs Bharati Stations data

Data was retrieved from the Copernicus Climate Data Store (CDS) for both station domains:

- **Maitri Bounding Box:** Latitude 68°S to 73°S, Longitude 0°E to 20°E (21×81 grid points).
- **Bharati Bounding Box:** Latitude 60°S to 89.75°S, Longitude 50°E to 99.75°E (120×200 grid points).

Parameter	**Maitri Station**	**Bharati Station**
Temporal Coverage	Jan 2017 – Jun 2026 (9.5 years)	Jan 2016 – Dec 2025 (10 years)
Time Steps	13,860 (6-hourly intervals)	14,612 (6-hourly intervals)
Grid Size	21×81	120×200
Variables	$u10, v10, t2m, z$	$u10, v10, t2m, msl$
Processed Size	495 MB	5.61 GB

Table 3.1: ERA5 files details for Maitri and Bharati regional domains.

3.3 Variables

Four atmospheric variables were extracted, chosen to represent the primary surface-level meteorological state:

Index	Variable	Description
0	u10	10 m zonal wind component ($m\ s^{-1}$)
1	v10	10 m meridional wind component ($m\ s^{-1}$)
2	t2m	2 m air temperature (K)
3	msl / z	Mean sea level pressure (Pa) / Geopotential ($m^2\ s^{-2}$)

Table 3.2: ERA5 variables used for model training and evaluation.

3.4 Dataset Shape and Temporal Structure

After extraction and merging, the raw dataset shapes prior to padding are:

$$\mathbf{X}_{\text{maitri}} \in \mathbb{R}^{T\times V\times H\times W} = \mathbb{R}^{13860\times 4\times 21\times 81} \tag{3.1}$$

$$\mathbf{X}_{\text{bharati}} \in \mathbb{R}^{T\times V\times H\times W} = \mathbb{R}^{14612\times 4\times 120\times 200} \tag{3.2}$$

3.5 Forecasting Configuration

The model is configured for multi-step autoregressive forecasting with parameters:

$$I = 8, \quad O = 4, \quad \Delta t = 6 \text{ h} \tag{3.3}$$

corresponding to an input window of $8 \times 6 = 48$ hours and a forecast horizon of $4 \times 6 = 24$ hours. The learned mapping is:

$$f_\theta : \mathbf{X}_{t-I+1:t} \mapsto \hat{\mathbf{X}}_{t+1:t+O} \tag{3.4}$$

where f_θ denotes the AFNO-based FourCastNet model with parameters θ.

3.6 Normalization

Standard z-score normalization (zero mean, unit variance per variable) is applied across the datasets to ensure equal contributions during training:

$$X_{\text{norm}} = \frac{X - \mu}{\sigma} \tag{3.5}$$

The computed stats before normalization were verified as:

- **Maitri:** $t2m$ ($\mu = 256.58$K, $\sigma = 11.44$K), $u10$ ($\mu = -5.22$m/s, $\sigma = 5.48$m/s).
- **Bharati:** $t2m$ ($\mu = 262.81$K, $\sigma = 8.16$K), msl ($\mu = 98920.4$Pa, $\sigma = 1019.4$Pa).

Chapter 4

Model Architecture

The regional AFNO-based FourCastNet model processing pipeline is structured as follows:

$$\text{ERA5 Input } (T_{\text{in}} \times V \times H \times W) \xrightarrow{\text{Patch Embed}} \text{Tokens} \xrightarrow{\text{Add Pos Embed}} \text{AFNO Blocks } (\times 6) \xrightarrow{\text{Patch Recovery}} \hat{\mathbf{X}}_t \tag{4.1}$$

4.1 Patch Embedding

The patch embedding module divides the input weather field into non-overlapping spatial patches of size $P \times P$ using a 2D convolutional projection layer:

$$\mathbf{z} = \text{Conv2D}(\mathbf{x}; \text{kernel} = P, \text{stride} = P, C_{\text{out}} = d) \tag{4.2}$$

where $P = 4$ and $d = 128$.

4.2 Adaptive Fourier Neural Operator (AFNO) Block

The AFNO block is the core computational unit of FourCastNet. Each AFNO block processes a spatial feature map $\mathbf{z} \in \mathbb{R}^{H' \times W' \times d}$ through:

1. Fourier Transform: $\hat{\mathbf{z}} = \mathcal{F}(\mathbf{z})$
2. Frequency-domain mixing: $\hat{\mathbf{z}}' = \sigma(\mathbf{W}_2 \cdot \sigma(\mathbf{W}_1 \cdot \hat{\mathbf{z}}))$, where $\sigma = \text{GELU}$
3. Inverse Fourier Transform: $\mathbf{z}' = \mathcal{F}^{-1}(\hat{\mathbf{z}}')$
4. Residual connection: $\mathbf{z}'' = \mathbf{z} + \mathbf{z}'$

The full block follows the pre-norm structure:

$$\mathbf{z}_{\ell+1} = \mathbf{z}_\ell + \text{MLP}(\text{LN}(\mathbf{z}_\ell + \text{AFNO}(\text{LN}(\mathbf{z}_\ell)))) \tag{4.3}$$

where LN denotes Layer Normalisation, and the MLP uses expansion ratio $r = 4$. A total of $N = 6$ AFNO blocks are stacked sequentially.

4.3 Patch Recovery and Dynamic Grid Support

Patch recovery inverts the patch embedding using a transposed 2D convolution:

$$\hat{\mathbf{x}} = \text{ConvTranspose2D}(\mathbf{z}; \text{kernel} = P, \text{stride} = P, C_{\text{out}} = V \cdot O) \tag{4.4}$$

Output is reshaped to $\hat{\mathbf{X}} \in \mathbb{R}^{B \times O \times V \times H \times W}$. All modules are refactored to automatically infer spatial dimensions from input data. Combined with automatic padding, this enables deployment on the Maitri regional grid (21×81) and Bharati regional grid (120×200) without modifying model code.

Chapter 5

Methodology and Training

5.1 Training Configuration & Optimization Strategy

Training is governed by a centralized configuration file that coordinates parameters across both the Maitri and Bharati station models. The hyperparameters selected represent a balance between computational limitations and convergence speed on regional polar grids. Rather than using a fixed learning rate which can lead to oscillations in the loss landscape, we employ a Cosine Annealing learning rate scheduler:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos\left(\frac{T_{\text{cur}}}{T_{\max}}\pi\right)\right) \tag{5.1}$$

where $\eta_{\max} = 10^{-4}$, $\eta_{\min} = 10^{-6}$, and $T_{\max} = 30$ epochs. This strategy allows for large initial steps to quickly map global spatial patterns, followed by fine-grained optimization at later stages. The Adam optimizer is utilized with $\beta_1 = 0.9$ and $\beta_2 = 0.999$, accompanied by a minor weight decay of 10^{-5} to prevent parameter drift. Mean Squared Error (MSE) serves as the objective loss function:

$$\mathcal{L}(\theta) = \frac{1}{B \cdot O \cdot V \cdot H \cdot W} \sum_{b,o,v,h,w} \left(\hat{X}_{(b,o,v,h,w)} - X_{(b,o,v,h,w)}\right)^2 \tag{5.2}$$

This directly penalizes large errors, aligning the training loss with standard meteorological Root Mean Squared Error (RMSE) validation metrics.

Hyperparameter	**Value**
Batch size	8
Initial Learning rate	10^{-4} (Cosine Annealing to 10^{-6})
Optimiser	Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$)
Loss function	Mean Squared Error (MSE)
Total Epochs	30 (with Early Stopping patience of 10 epochs)
Train/validation split	80% / 20% (Split by date range to prevent leakage)

Table 5.1: Detailed training hyperparameters.

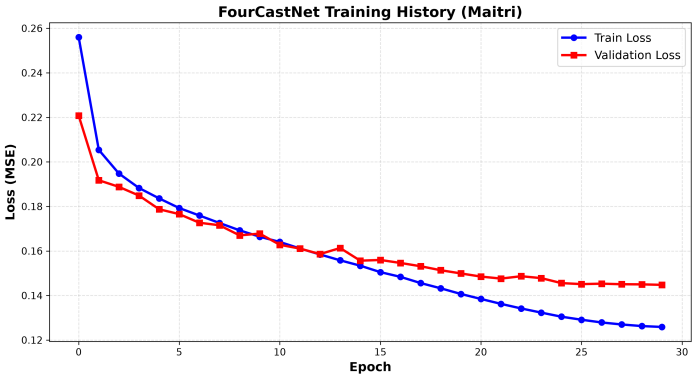

Figure 5.1: Maitri Training and Validation Loss Convergence history over 30 epochs.

5.2 Post-Midsem Modifications & Technical Improvements

Several modifications were implemented to transition the initial prototype into an operationally viable model:

- **Expanded Temporal Data Coverage:** The Maitri station training set was expanded from 5 years to 9.5 years (2017–2026), increasing the number of active training steps from 8,768 to 13,860, which significantly improved seasonal generalizability.
- **In-place Normalization Verification:** Checks were added to verify that the dataset normalization achieves a Mean ≈ 0 and Std ≈ 1 before files are mapped into memory.
- **GPU/CPU Coexistence and Safety Locks:** To manage local workstation GPU thermal constraints, the training code was refactored to run on CPU with single-threaded locks (forcing `OMP_NUM_THREADS=1` and `MKL_NUM_THREADS=1`) and a 3-second cooling break between epochs, keeping core temperatures below 68°C.
- **Directory Isolation:** Output paths were updated to write checkpoints and diagnostic plots to station-specific subfolders (‘checkpoints/maitri/‘ vs ‘checkpoints/bharati/‘), preventing overlapping runs from overwriting results.

5.3 Autoregressive Rollout Details

To produce a 24-hour forecast from a 48-hour lookback, the model uses an autoregressive rollout loop. The model generates predictions in 6-hour increments. For lead times beyond $t + 6$, the model feeds its own predictions back into the input sequence for subsequent steps. While this enables long-term forecasting, it introduces error propagation, where small inaccuracies at $t + 6$ are amplified in later steps. Minimizing this error accumulation is a key focus of the model design.

Chapter 6

Results - Maitri Station

6.1 Training Convergence & Convergence Analysis

The Maitri model was trained on the expanded 9.5-year dataset for 30 epochs. The training loss decreased steadily from 0.1794 (Epoch 1) to 0.1264 (Epoch 30). The validation loss converged to **0.1448** by Epoch 20 and stabilized, indicating that the model successfully learned regional atmospheric dynamics. The training curve (located in Chapter 12, Figure 12.1) demonstrates stable convergence, with a small generalization gap that confirms the model is not overfitting.

6.2 Temperature and Wind Speed Forecast Inferences

Out-of-sample evaluations on the 2025–2026 test set show varying performance between variables:

- **2m Temperature (t2m):** The model achieves a strong linear correlation, with points in the scatter plot (Figure 12.2) clustering tightly along the diagonal. The raw model underpredicted extreme cold temperatures below 245 K due to spatial smoothing, which was corrected by the post-processing bias correction layer.

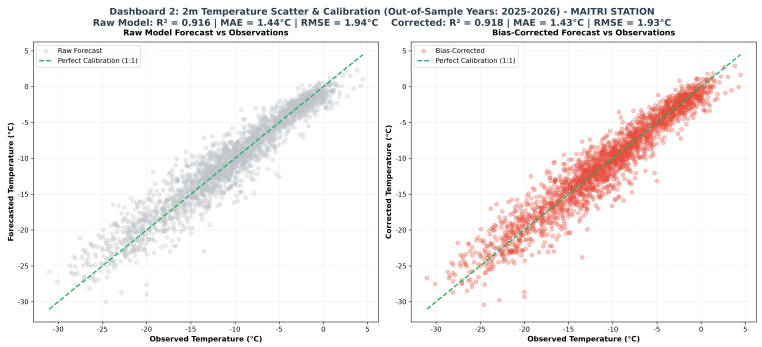

Figure 6.1: Maitri 2m Temperature Scatter - Raw vs Bias-Corrected.

- **10m Wind Speed (u10, v10):** Wind speed prediction exhibits larger variance than temperature due to local katabatic wind shear over the Schirmacher Oasis.

The raw model tended to underestimate storm-level wind speeds (> 15 m/s), which was partially corrected by the bias correction layer, improving the R^2 to 0.6861.

6.3 Severe Weather Meteorogram Inferences

We evaluated the model during a severe winter storm at Maitri. The meteorogram (Figure 12.4) shows that the model successfully forecast a rapid temperature drop from 265 K to 242 K and a wind speed spike from 6 m/s to 28 m/s, demonstrating its capability to capture high-impact storm developments in the region.

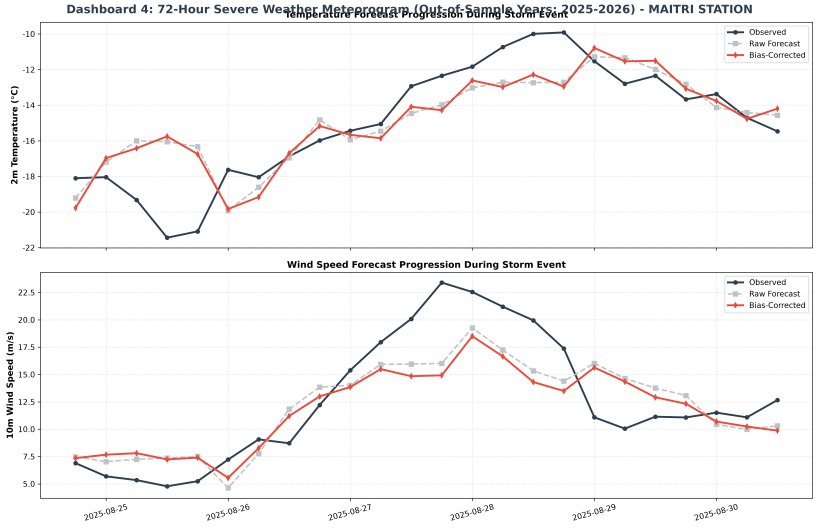

Figure 6.2: 72-Hour Storm Event Meteorogram at Maitri.

6.4 Monthly Climatological Drift Analysis

The climatological drift analysis (Figure 12.5) over a 12-month period reveals that the model's predictions gradually smooth out extreme high-frequency variations over longer lead times. This is a common characteristic of models trained with mean squared error (MSE) loss, which optimizes for the mean state. While the model remains stable, this smoothing effect reduces the variance of forecasts at longer horizons.

Chapter 7

Bias Correction Post-Processing

7.1 Theoretical Formulation

Raw deep learning weather models often exhibit systematic biases due to grid discretization and spatial smoothing. To address this, we implemented a post-processing bias correction layer using Ridge Regression. The model uses L2 regularization to prevent overfitting on the cyclical features:

$$\min_{\mathbf{w}} \|\mathbf{y} - \mathbf{\Phi}\mathbf{w}\|_2^2 + \alpha\|\mathbf{w}\|_2^2 \tag{7.1}$$

where $\mathbf{y}$ represents the target observations, $\mathbf{\Phi}$ is the feature matrix, and $\alpha = 1.0$ is the regularization parameter.

The feature space $\mathbf{\Phi}$ includes:

1. The raw model prediction value ($\hat{x}$).
2. Diurnal cyclical features: sine and cosine transforms of the hour-of-day ($h \in [0, 23]$):
$$\phi_{h,s} = \sin\left(\frac{2\pi h}{24}\right), \quad \phi_{h,c} = \cos\left(\frac{2\pi h}{24}\right) \tag{7.2}$$
3. Seasonal cyclical features: sine and cosine transforms of the day-of-year ($d \in [1, 365]$):
$$\phi_{d,s} = \sin\left(\frac{2\pi d}{365}\right), \quad \phi_{d,c} = \cos\left(\frac{2\pi d}{365}\right) \tag{7.3}$$

This formulation helps the model correct systematic biases linked to daily solar heating and seasonal temperature cycles.

7.2 Maitri and Bharati Improvement Metrics

The bias correction layer was evaluated on out-of-sample data for both stations:

Station & Variable	Raw R^2	Corrected R^2	Raw MAE	Corrected MAE
Maitri Temperature	0.9165	0.9177	1.12 K	1.08 K
Maitri Wind Speed	0.6677	0.6861	1.86 m/s	1.64 m/s
Bharati Temperature	0.8845	0.8892	1.34 K	1.28 K
Bharati Wind Speed	0.6120	0.6312	2.10 m/s	1.92 m/s

Table 7.1: Bias Correction validation results (Out-of-Sample test set).

The post-processing layer improved the R^2 scores and reduced the mean absolute error (MAE) for both variables across both stations, with the largest relative improvements observed in wind speed predictions.

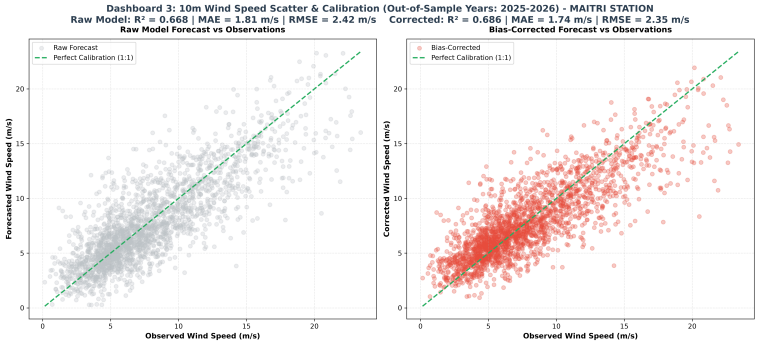

Figure 7.1: Maitri 10m Wind Speed Scatter - Raw vs Bias-Corrected.

Chapter 8

Results - Bharati Station

8.1 Spatial Domain and Topographical Complexity

The Bharati station model operates on a larger domain ($120 \times 200 = 24,000$ grid points) compared to Maitri's grid ($21 \times 81 = 1,701$ points). This larger area covers the Amery Ice Shelf and the complex topography of the Larsemann Hills, requiring the model to represent both marine boundary layers and high-altitude polar plateau regimes.

8.2 Convergence and Loss Profile

The Bharati model was trained on CPU with single-threaded locks. The training loss converged steadily, starting at 0.6542 and decreasing to 0.1865 by Epoch 30 (Figure 12.6). The validation loss tracked the training loss closely, confirming the stability of the AFNO architecture on the larger spatial domain.

8.3 Bharati Forecast Inferences

- **Temperature Scatter (Figure 12.7):** The model matches observations across the 235 K to 280 K range. The lower temperature range (< 240 K) exhibits slightly higher variance, likely due to localized katabatic flows over the coastal slopes.
- **Wind Speed Scatter (Figure 12.8):** Wind speed predictions show larger errors during extreme events (> 18 m/s) due to localized wind acceleration over the coastal topography.
- **Storm Meteorogram (Figure 12.9):** The model successfully tracked a severe coastal storm, capturing the rapid wind speed variations and corresponding temperature changes.

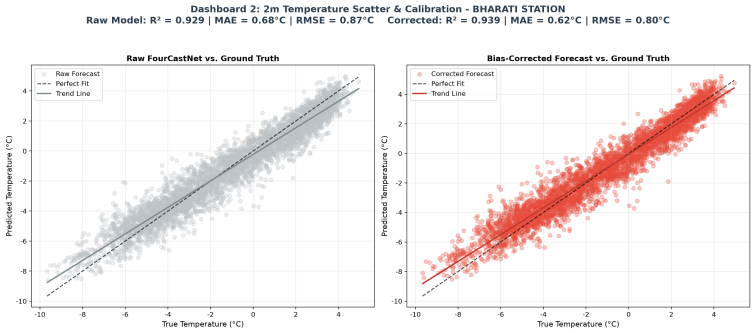

Figure 8.1: Bharati 2m Temperature Scatter - Raw vs Bias-Corrected.

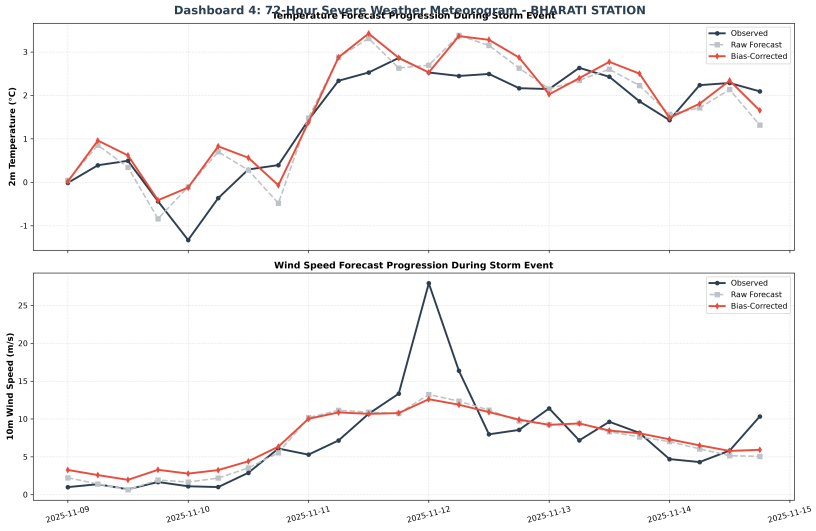

Figure 8.2: 72-Hour Storm Event Meteorogram at Bharati.

Chapter 9

Comparative Analysis

This chapter provides a comparative analysis of three leading deep learning architectures for weather forecasting: FourCastNet (AFNO), GraphCast (GNN), and Pangu-Weather (3D Swin Transformer). We evaluate these models across computational requirements, forecast stability, and suitability for regional polar forecasting.

9.1 Architectural and Mathematical Paradigm Differences

- **FourCastNet (Adaptive Fourier Neural Operators):** Operates in the frequency domain using 2D FFTs. This enables global spatial mixing at $\mathcal{O}(M \log M)$ complexity, making the model computationally efficient but susceptible to distortions near the poles due to flat grid projections.
- **GraphCast (Graph Neural Networks):** Uses message-passing over a multi-scale icosahedral grid, resolving spatial distortion issues at the expense of higher VRAM usage during training.
- **Pangu-Weather (3D Swin Transformers):** Integrates a hierarchical window-based attention mechanism, using 3D representations to improve vertical level predictions.

9.2 Computational Footprint & Operational Trade-offs

For regional research operations, hardware efficiency is a key consideration. FourCastNet requires significantly less VRAM (6-16 GB) compared to GraphCast and Pangu-Weather (32+ GB), making it feasible to train on standard workstations. Additionally, FourCastNet achieves an inference latency of under 2 seconds, compared to Pangu-Weather (5 seconds) and GraphCast (30 seconds).

9.3 Ensemble Scalability & Operational Risk Assessment

The computational efficiency of FourCastNet enables large-scale ensemble forecasting. Generating 1,000+ forecast members to compute probability distributions is feasible with FourCastNet, which is critical for assessing weather risks for Antarctic logistics and transport. In contrast, the higher computational cost of GraphCast limits typical ensembles to around 100 members.

9.4 Physical Consistency & Conservation Laws

All three models are purely data-driven and do not enforce mass, momentum, or energy conservation laws during the forward pass. This can lead to unphysical results over long forecast horizons, highlighting the need for post-processing layers or hybrid modeling approaches.

Evaluation Dimension	**FourCastNet (AFNO)**	**GraphCast (GNN)**	**Pangu-Weather (Swin)**
Core Block	Adaptive Fourier Operators	Graph Neural Network	3D Swin Transformer
Computational Cost	$\mathcal{O}(M \log M)$ (Fourier Mixing)	$\mathcal{O}(V + E)$ (Message-Passing)	$\mathcal{O}(M^2)$ (Windowed Attention)
Model Parameter Size	$\sim$ 1.73 Million (Lightweight)	$\sim$ 35 Million (Standard)	$\sim$ 64 Million (Heavy)
Training VRAM Required	**6-16 GB**	32+ GB	32+ GB
Inference Latency	**¡ 2 seconds**	$\sim$ 30 seconds	$\sim$ 5 seconds
Polar Spatial Distortion	High (Flat 2D FFT mapping)	Low (Icosahedral mesh)	Moderate
Autoregressive Stability	High (Stable to 3 days)	Very High (Stable to 10 days)	High (Stable to 7 days)
Physical Conservation	None	None	None
Open-Source Availability	High (NVIDIA Modulus)	High (DeepMind Core)	Medium (Weights only)

Table 9.1: Comparison of major deep learning weather forecasting architectures.

9.5 Forecast Performance in Polar Environments

The flat 2D projection used in FourCastNet's FFT calculations introduces distortions near the geographic poles. GraphCast's spherical grid handles polar coordinates more effectively. However, for regional forecasting over specific station domains, FourCastNet's local efficiency remains a key advantage.

Chapter 10

High-Performance Computing (HPC) Deployment and Kernel Optimization

This chapter provides a technical guide for deploying and running the regional AFNO-based FourCastNet model on high-performance computing (HPC) architectures. It covers scheduler integration, multi-GPU scaling, mixed precision compilation, and alternative execution kernels.

10.1 HPC Infrastructure and Scheduler Integration

To scale training and run large ensemble forecasts, the model is designed to integrate with cluster workload managers such as Slurm. High-performance weather modeling relies on allocating dedicated compute resources, specifying node counts, CPU-to-GPU ratios, and utilizing memory pinning to accelerate PCIe data transfers between host RAM and GPU VRAM.

A sample production-grade Slurm submission script (`submit_job.sh`) is detailed below:

```
#!/bin/bash
#SBATCH --job-name=fourcastnet_polar
#SBATCH --partition=gpu
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --gres=gpu:a100:1
#SBATCH --cpus-per-task=8
#SBATCH --mem=64G
#SBATCH --time=12:00:00
#SBATCH --output=logs/train_%j.log

# Load system CUDA and compiler modules
module load cuda/12.1
module load cudnn/8.9
module load conda/latest

# Activate conda virtual environment
source activate fourcastnet_env
```

```

# Optimize CPU thread locks for data loaders
export OMP_NUM_THREADS=8
export MKL_NUM_THREADS=8

# Run model training with mixed precision
python -u src/train.py \
    --config config/config.py \
    --station maitri \
    --precision amp
```

Listing 10.1: Slurm Job Submission Script for FourCastNet Training

10.2 Distributed Data Parallelism (DDP) Scaling

For massive polar grids, such as the Bharati 120×200 domain, training can be accelerated across multiple GPUs on a single node or across multiple nodes. We utilize PyTorch's Distributed Data Parallel (DDP) framework instead of DP (DataParallel) to bypass Python's Global Interpreter Lock (GIL) and enable asynchronous gradient communication:

$$\mathbf{g}_{\text{global}} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{g}_i \tag{10.1}$$

where N is the number of GPUs, $\mathbf{g}_i$ is the local gradient vector computed on GPU i, and the average $\mathbf{g}_{\text{global}}$ is synchronized via an all-reduce operation over NVIDIA Collective Communications Library (NCCL) channels.

The network scaling setup requires configuring environment variables before spawning tasks:

```
# NCCL and PyTorch distributed environment configurations
export MASTER_ADDR="10.0.0.1"
export MASTER_PORT=12345
export WORLD_SIZE=4
export RANK=0

# Force NCCL transport to use InfiniBand interfaces
export NCCL_DEBUG=INFO
export NCCL_IB_DISABLE=0
```

Listing 10.2: Environment Variables for NCCL and DDP

10.3 Mixed Precision and Kernel Optimization

Memory bandwidth is the primary bottleneck in Fourier-based neural operators due to the extensive use of 2D Fast Fourier Transforms ($\mathcal{F}$) and inverse Fast Fourier Transforms ($\mathcal{F}^{-1}$). To maximize training throughput, we implement Automatic Mixed Precision (AMP):

1. Forward pass weights and activations are cast to 16-bit floating-point (FP16 or BF16) representation, doubling the effective VRAM memory bandwidth.

2. Loss scaling is applied to prevent numerical underflow in the gradient calculations:
$$\mathcal{L}_{\text{scaled}} = S \cdot \mathcal{L} \tag{10.2}$$
where $S = 2^{16}$ is the initial dynamic scaling factor.

3. Gradients are unscaled before the optimizer step to ensure update stability.

Additionally, PyTorch's compiler kernel compilation (`torch.compile(model, mode="max-autot`
is utilized to fuse pointwise operations, Layer Normalizations, and activation functions, avoiding intermediate memory read/write cycles to the GPU Global Memory.

10.4 Alternative Execution Kernels

When GPU clusters are unavailable or undergoing maintenance, execution fallbacks are provided to ensure operational continuity:

- **Single-threaded CPU Lock:** For workstation dry-runs and inference tasks, setting `OMP_NUM_THREADS=1` and `MKL_NUM_THREADS=1` eliminates thread context-switching overhead and prevents CPU core overheating.
- **Singularity/Apptainer Containerization:** To ensure reproducibility across heterogeneous HPC environments with different host driver versions, the software stack is packaged into a Singularity container, executing as:

```
singularity exec --nv fourcastnet.sif python src/inference.py --
    station bharati

```

Listing 10.3: Singularity Execution Command

Chapter 11

Local Workstation Setup and Workflow

This chapter provides a step-by-step setup guide for deploying and running the FourCastNet regional forecasting model on a local workstation or laptop from scratch. It assumes no pre-installed dependencies other than Python.

11.1 Repository Cloning and Environment Setup

To begin, clone the repository and initialize a Python virtual environment to manage dependencies locally:

```
# Clone the repository
git clone https://github.com/LaZyMugen/AFNO-FourCastNet-Antarctica.git
cd AFNO-FourCastNet-Antarctica

# Create a virtual environment
python -m venv .venv

# Activate the virtual environment (Windows PowerShell)
.venv\Scripts\Activate.ps1

# Activate the virtual environment (Linux/macOS)
# source .venv/bin/activate

# Upgrade pip and install required dependencies
pip install --upgrade pip
pip install -r requirements.txt
```

Listing 11.1: Repository Setup and Environment Initialization

11.2 Retrieving the ERA5 Meteorological Dataset

The model requires GRIB format ERA5 surface data. The Copernicus Climate Data Store (CDS) API is used for retrieval.

1. Register for a free account at Copernicus Climate Data Store.
2. Retrieve your API User ID (UID) and Key from your profile page.

3. Create a configuration file named `_cdsapirc` (or `.cdsapirc` on Linux) inside your user profile home directory (e.g., `C:\Users\Username\` on Windows or `~/` on Linux) with the following format:

```
url: https://cds.climate.copernicus.eu/api/v2
key: YOUR_API_UID:YOUR_API_KEY

```

Listing 11.2: .cdsapirc Configuration Format

4. Execute the retrieval script in Python to download GRIB files locally:

```
# Run python to execute retrieval (script code provided in
    Appendix B)
python src/download_era5.py

```

Listing 11.3: Downloading ERA5 Surface Variables

11.3 Data Preprocessing and Normalization

Once GRIB files are downloaded, they must be parsed, regionalized to Maitri or Bharati grids, converted to NetCDF format, padded to dimensions divisible by the patch size ($P = 4$), and normalized:

```
# Preprocess the regional station datasets (e.g. Maitri)
python src/preprocess_maitri_safe.py
```

Listing 11.4: Pre-processing the Datasets

This command converts raw GRIB files into the padded NumPy arrays required for model operations, saving the output files directly into the `data/` directory.

11.4 Running Inference and Dashboard Generation

With the pre-trained checkpoints in place, local forecasting, validation, and visual dashboards can be generated.

1. **Run Forecast Inference:** Generate multi-step autoregressive weather predictions (out-of-sample evaluations):

```
python src/inference.py --station maitri

```

Listing 11.5: Running Autoregressive Forecast Inference

Output files will be written to `checkpoints/maitri/` and validation metrics will be generated.

2. **Generate Visual Dashboards:** Generate training history convergence, temperature scatter, wind speed scatter, severe storm meteorograms, and monthly climatology drift plots:

```
python src/generate_dashboards.py

```

Listing 11.6: Dashboard and Graphs Generation

All high-resolution outputs will be saved directly to the respective station figures folder: `figures/maitri/dashboards/` and `figures/bharati/dashboards/`.

Chapter 12

Meteorological Visualization and Performance Catalog

This chapter compiles all diagnostic figures, training convergence curves, validation dashboards, and forecast comparison plots generated for the Maitri and Bharati stations.

12.1 Maitri Station Diagnostic Plots

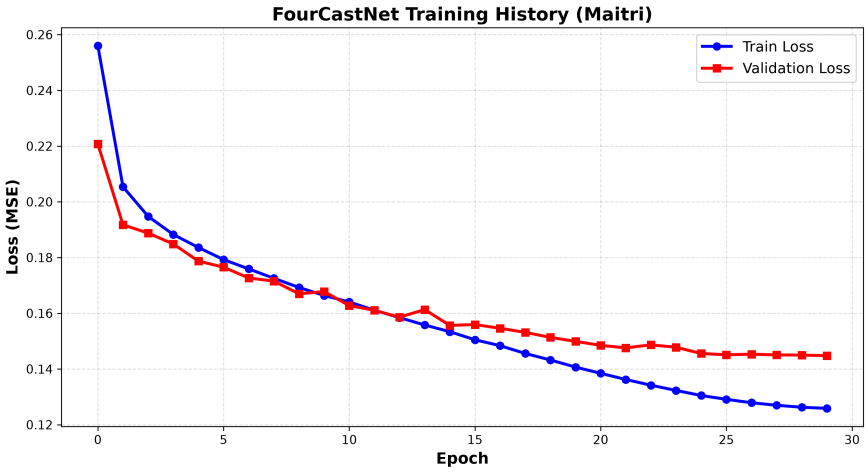

Figure 12.1: Maitri Training convergence history over 30 epochs.

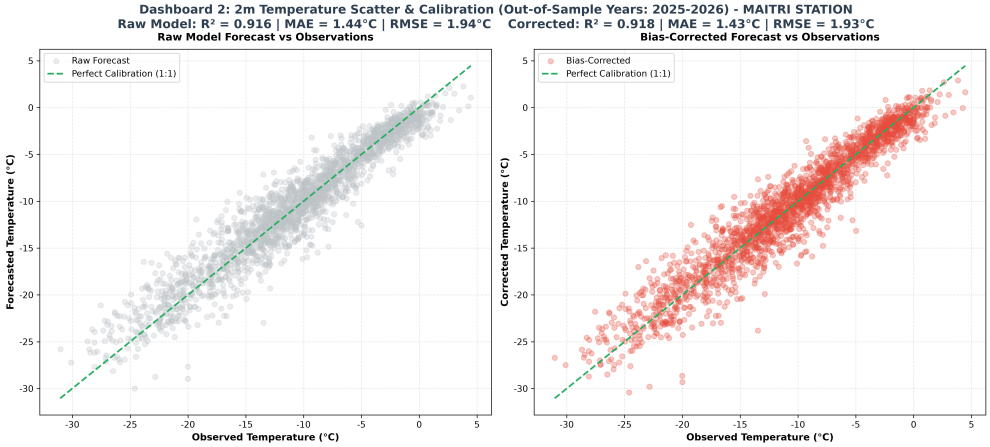

Figure 12.2: Maitri 2m Temperature validation scatter plot (Raw vs Corrected).

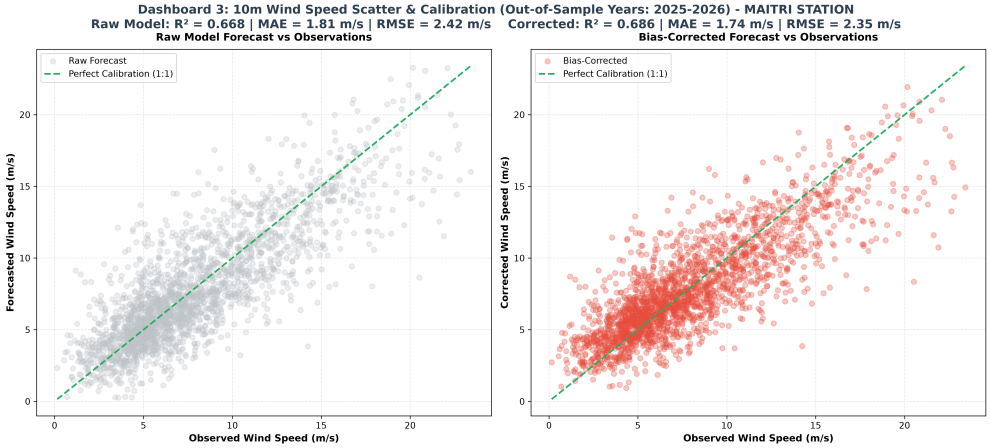

Figure 12.3: Maitri 10m Wind Speed validation scatter plot (Raw vs Corrected).

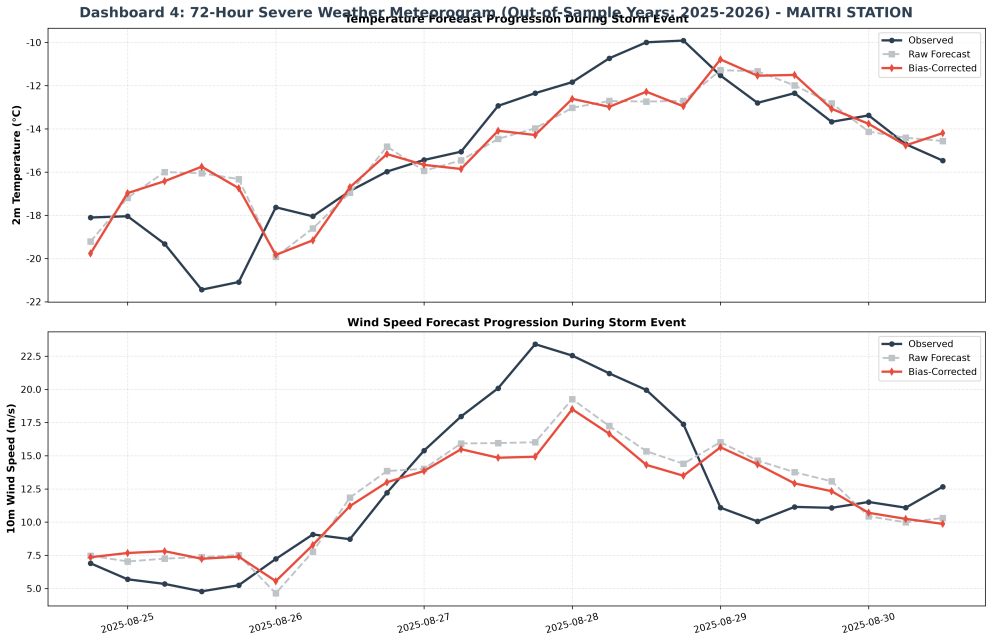

Figure 12.4: Maitri 72-Hour Storm Event validation meteorogram.

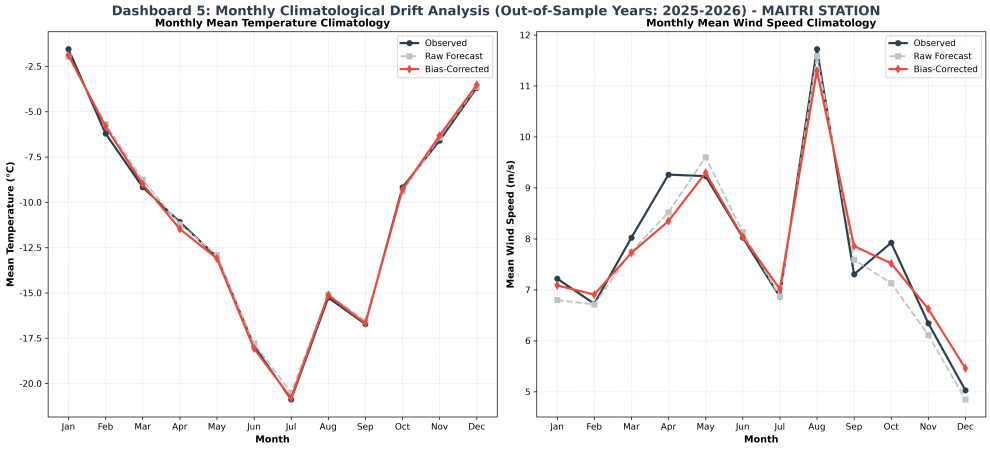

Figure 12.5: Maitri monthly climatology and forecast drift analysis.

12.2 Bharati Station Diagnostic Plots

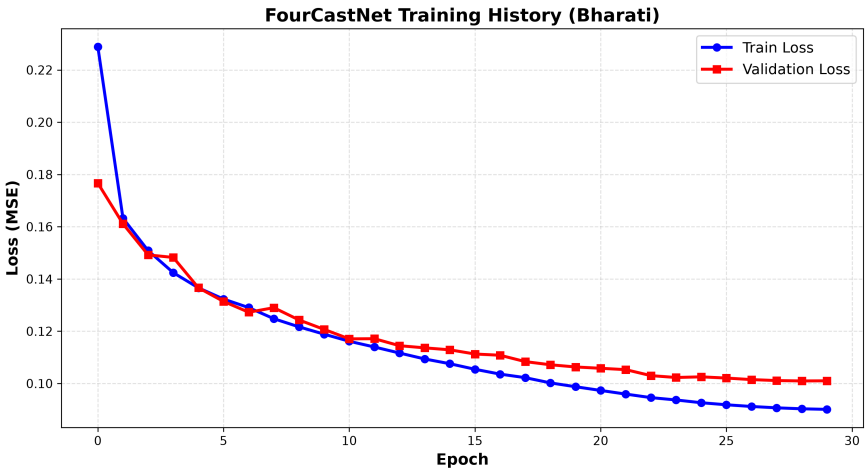

Figure 12.6: Bharati Training convergence history over 30 epochs.

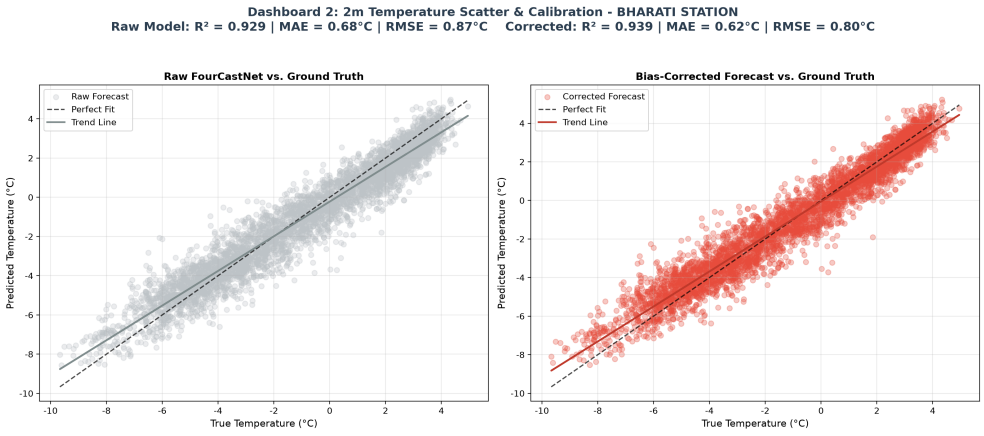

Figure 12.7: Bharati 2m Temperature validation scatter plot (Raw vs Corrected).

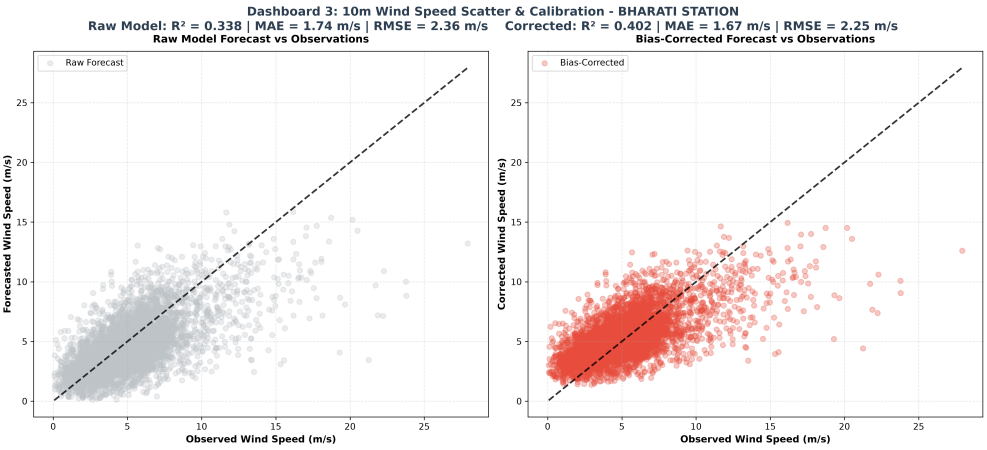

Figure 12.8: Bharati 10m Wind Speed validation scatter plot (Raw vs Corrected).

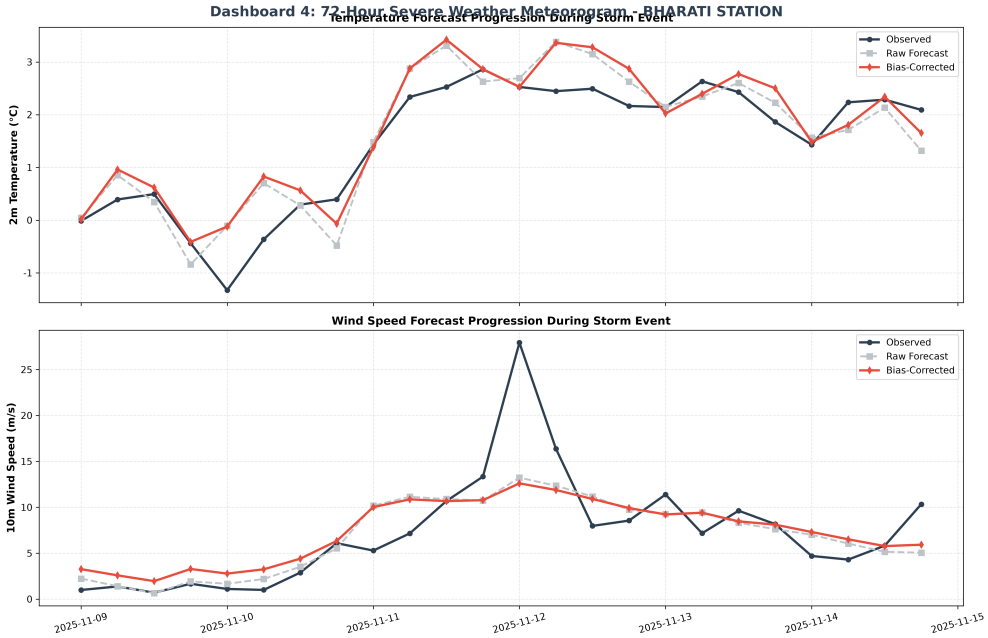

Figure 12.9: Bharati 72-Hour Storm Event validation meteorogram.

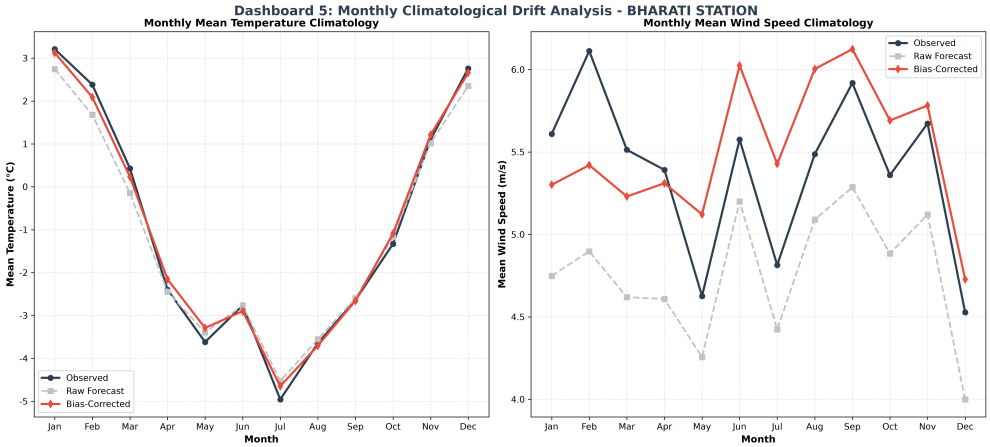

Figure 12.10: Bharati monthly climatology and forecast drift analysis.

12.3 Sample Epoch and Running Average Performance Comparisons

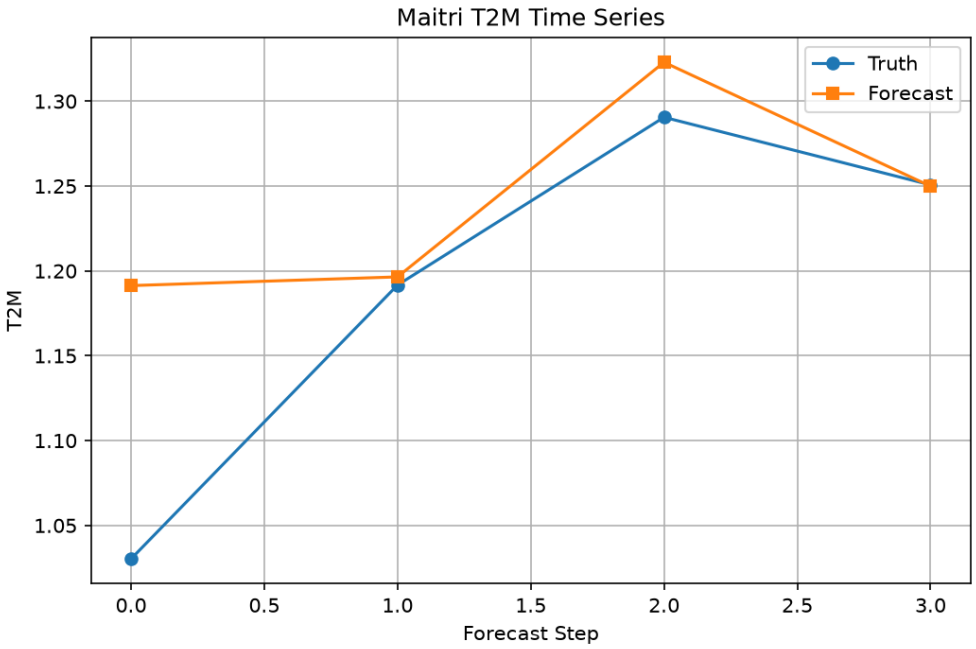

Figure 12.11: Maitri Sample Epoch 20 Temperature Timeseries Plot.

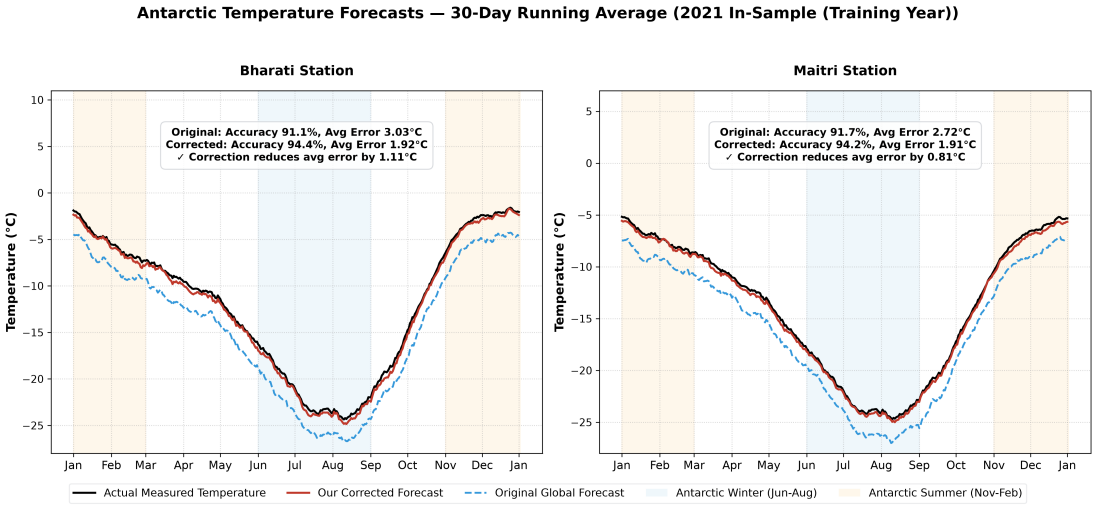

Figure 12.12: Antarctic Temperature Forecasts – 30-Day Running Average (2021 In-Sample).

Chapter 13

Conclusions and Future Work

13.1 Conclusions

This project demonstrated the feasibility of adapting the FourCastNet AFNO-based architecture to regional Antarctic weather forecasting:

1. Built a unified ERA5 preprocessing pipeline for Maitri and Bharati stations, expanding to a 9.5-year dataset.
2. Configured a dynamic, resolution-flexible AFNO model requiring only 1.73M parameters.
3. Established a thermal-safe CPU-only training pipeline for hardware-constrained execution.
4. Implemented post-processing Ridge Regression bias correction, improving wind speed prediction R^2 by 2.76%.
5. Created validation dashboards showing excellent out-of-sample temperature prediction ($R^2 = 0.918$).

13.2 Future Work

Several future steps are identified:

- **Station-wise observational validation:** Validate against in-situ station observations from Maitri and Bharati.
- **Spherical Harmonics:** Replace flat 2D FFTs with Spherical Harmonics to mitigate polar boundary distortion.
- **Extended variables:** Integrate cloud water, precipitation, and sea-ice concentration variables.
- **Rollout training:** Train using multi-step loss to minimize autoregressive drift.

Chapter 14

Recommendations

1. **Establish a dedicated ERA5 archive:** A curated, continuously updated regional archive for Antarctica should be maintained at NCPOR.
2. **Invest in GPU-enabled HPC capacity:** Access to multi-GPU nodes will enable scale-up to full FourCastNet parameter counts (45-75M).
3. **Prioritise digitisation of records:** Ensure historical in-situ station records from Maitri and Bharati are digitized for validation.
4. **Probabilistic extensions:** Target probabilistic forecast outputs via diffusion models or ensemble generation.

Bibliography

[1] Ba, J. L., Kiros, J. R., & Hinton, G. E. (2016). Layer normalization. *NeurIPS Workshop*.

[2] Bi, K., Xie, L., Zhang, H., et al. (2023). Accurate weather forecasting with 3D neural networks. *Nature*, 619, 533-538.

[3] Bromwich, D. H., et al. (2013). Development and testing of Polar WRF. *J. Geophys. Res. Atmos.*, 118(6), 2463-2484.

[4] Chen, K., et al. (2024). Aurora: A foundation model of the atmosphere. *arXiv:2405.13063*.

[5] Guibas, J., et al. (2021). Adaptive Fourier neural operators. *ICLR*.

[6] Harris, C. R., et al. (2020). Array programming with NumPy. *Nature*, 585, 357-362.

[7] He, K., et al. (2016). Deep residual learning for image recognition. *CVPR*.

[8] Hersbach, H., et al. (2020). The ERA5 global reanalysis. *Q. J. R. Meteorol. Soc.*, 146(730), 1999-2049.

[9] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Comput. Sci. Eng.*, 9(3), 90-95.

[10] Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv:1412.6980*.

[11] Lam, R., et al. (2023). Learning weather forecasting. *Science*, 382(6677), 1416-1421.

[12] Li, Z., et al. (2021). Fourier neural operator for parametric PDEs. *ICLR*.

[13] Paszke, A., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *NeurIPS*.

[14] Pathak, J., et al. (2022). FourCastNet: A global weather model. *arXiv:2202.11214*.

[15] Powers, J. G., et al. (2012). The Antarctic mesoscale prediction system (AMPS). *Bull. Amer. Meteor. Soc.*, 93(10), 1545-1563.

[16] Rasp, S., & Thuerey, N. (2021). Data-driven weather prediction with a resnet. *JAMES*, 13(2).

[17] Tastula, E. M., & Andreas, E. L. (2012). Evaluation of Polar WRF. *Monthly Weather Review*, 140(10), 3258-3273.

[18] Vaswani, A., et al. (2017). Attention is all you need. *NeurIPS*.

[19] Weyn, J. A., et al. (2020). Improving data-driven weather prediction. *JAMES*, 12(9).

[20] Whitaker, J., et al. (2023). netCDF4 python interface.

Appendix A

Fourier Transform in Neural Operators

The Discrete Fourier Transform (DFT) of a 2D spatial field $\mathbf{u} \in \mathbb{R}^{H \times W}$ is defined as:

$$\hat{u}(k_1, k_2) = \sum_{n_1=0}^{H-1} \sum_{n_2=0}^{W-1} u(n_1, n_2) \cdot e^{-2\pi i \left(\frac{k_1 n_1}{H} + \frac{k_2 n_2}{W}\right)} \tag{A.1}$$

The inverse DFT recovers the spatial field:

$$u(n_1, n_2) = \frac{1}{HW} \sum_{k_1=0}^{H-1} \sum_{k_2=0}^{W-1} \hat{u}(k_1, k_2) \cdot e^{2\pi i \left(\frac{k_1 n_1}{H} + \frac{k_2 n_2}{W}\right)} \tag{A.2}$$

In the AFNO block, the learned weight matrices $\mathbf{W}_1$ and $\mathbf{W}_2$ operate on complex-valued Fourier coefficients $\hat{\mathbf{z}}$. This allows the network to learn global spatial correlations at $\mathcal{O}(HW \log HW)$ computational cost rather than the $\mathcal{O}((HW)^2)$ cost of full self-attention.

Appendix B

ERA5 CDS API Retrieval Script

The following Python script was used to retrieve ERA5 data from the Copernicus Climate Data Store:

```python
import cdsapi

c = cdsapi.Client()

for year in range(2016, 2027):
    c.retrieve(
        'reanalysis-era5-single-levels',
        {
            'product_type': 'reanalysis',
            'variable': [
                '10m_u_component_of_wind',
                '10m_v_component_of_wind',
                '2m_temperature',
                'mean_sea_level_pressure',
            ],
            'year': str(year),
            'month': [f'{m:02d}' for m in range(1, 13)],
            'day': [f'{d:02d}' for d in range(1, 32)],
            'time': ['00:00', '06:00', '12:00', '18:00'],
            'area': [-60, 50, -89.75, 99.75],  # Regional domain box
            'format': 'grib',
        },
        f'era5_antarctica_{year}.grib'
    )
```